

Aula Detalhada — Números Binários, Operações, Sinal e Complemento de 2

1. O que significa estar em base 2?

No sistema decimal, usamos **10 símbolos**:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Por isso ele é chamado de **base 10**.

No sistema binário, usamos apenas **2 símbolos**:

0 e 1

Por isso ele é chamado de **base 2**.

Cada posição em um número binário representa uma potência de 2.

2. Pesos das posições em binário

Em decimal, o número 347 significa:

3 centenas + 4 dezenas + 7 unidades

Ou seja:

$3 \times 100 + 4 \times 10 + 7 \times 1$

Em binário, a lógica é parecida, mas os pesos são potências de 2.

Da direita para a esquerda:

... 128 64 32 16 8 4 2 1

Ou seja:

$2^7 \quad 2^6 \quad 2^5 \quad 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0$

Exemplo:

1011_2

Pesos:

1	0	1	1
8	4	2	1

Cálculo:

$$1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 11$$

Então:

$$1011_2 = 11_{10}$$

3. Convertendo binário para decimal

Passos:

1. Escreva os pesos das posições.
2. Multiplique cada bit pelo seu peso.
3. Some os pesos onde o bit é 1.

Exemplo:

110101_2

Pesos:

1	1	0	1	0	1
32	16	8	4	2	1

Cálculo:

$$\begin{aligned} &1 \times 32 + 1 \times 16 + 0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 \\ &= 32 + 16 + 4 + 1 \\ &= 53 \end{aligned}$$

Logo:

$$110101_2 = 53_{10}$$

4. Convertendo decimal para binário

Existem dois métodos comuns.

4.1 Método das divisões por 2

Exemplo: converter 13 para binário.

Divida por 2 e anote os restos:

$13 \div 2 = 6$, resto 1
 $6 \div 2 = 3$, resto 0
 $3 \div 2 = 1$, resto 1
 $1 \div 2 = 0$, resto 1

Agora leia os restos de baixo para cima:

1101

Então:

$13_{10} = 1101_2$

4.2 Método dos pesos

Exemplo: converter 45 para binário.

Pegue os pesos:

32 16 8 4 2 1

Veja quais somam 45:

$45 = 32 + 8 + 4 + 1$

Então:

32 16 8 4 2 1
1 0 1 1 0 1

Logo:

$45_{10} = 101101_2$

4.3 Parte fracionária em binário

Até aqui, trabalhamos principalmente com números inteiros. Mas números binários também podem ter parte fracionária.

Em português, normalmente falamos em vírgula:

$101,101_2$

Em livros e ferramentas de computação, também é comum aparecer com ponto:

101.101_2

As duas escritas acima representam a mesma ideia: uma parte inteira antes da vírgula/ponto e uma parte fracionária depois.

4.3.1 Pesos antes e depois da vírgula

Antes da vírgula, os pesos crescem para a esquerda:

8	4	2	1
2^3	2^2	2^1	2^0

Depois da vírgula, os pesos diminuem para a direita:

$1/2$	$1/4$	$1/8$	$1/16$	$1/32$
2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}

Então:

$101,101_2$

significa:

$1 \times 4 + 0 \times 2 + 1 \times 1 + 1 \times 1/2 + 0 \times 1/4 + 1 \times 1/8$

Calculando:

$4 + 1 + 0,5 + 0,125 = 5,625$

Logo:

$101,101_2 = 5,625_{10}$

4.3.2 Convertendo binário fracionário para decimal

Passos:

1. Separe a parte inteira da parte fracionária.
2. Converta a parte inteira normalmente.
3. Na parte fracionária, use os pesos $1/2$, $1/4$, $1/8$, $1/16$...
4. Some tudo.

Exemplo:

$$11,01_2$$

Parte inteira:

$$11_2 = 3_{10}$$

Parte fracionária:

$$0,01_2 = 0 \times 1/2 + 1 \times 1/4 = 0,25$$

Resultado:

$$11,01_2 = 3,25_{10}$$

4.3.3 Convertendo decimal fracionário para binário

Para a parte inteira, use divisão por 2 ou pesos, como antes.

Para a parte fracionária, use multiplicações por 2:

1. Multiplique a parte fracionária por 2.
2. A parte inteira do resultado vira o próximo bit.
3. Continue usando a nova parte fracionária.
4. Pare quando a fração virar 0 ou quando atingir a precisão pedida.

Exemplo: converter $0,625_{10}$ para binário.

$$\begin{array}{lcl} 0,625 \times 2 = 1,25 & \rightarrow & \text{bit 1, sobra } 0,25 \\ 0,25 \times 2 = 0,5 & \rightarrow & \text{bit 0, sobra } 0,5 \\ 0,5 \times 2 = 1,0 & \rightarrow & \text{bit 1, sobra } 0 \end{array}$$

Lendo os bits na ordem em que apareceram:

$$0,625_{10} = 0,101_2$$

Conferindo:

$$\begin{aligned} 0,101_2 &= 1 \times 1/2 + 0 \times 1/4 + 1 \times 1/8 \\ &= 0,5 + 0,125 \\ &= 0,625 \end{aligned}$$

4.3.4 Exemplo com parte inteira e fracionária

Converter:

$$6,25_{10}$$

Parte inteira:

$$6_{10} = 110_2$$

Parte fracionária:

$$\begin{aligned} 0,25 \times 2 &= 0,5 \rightarrow \text{bit } 0, \text{ sobra } 0,5 \\ 0,5 \times 2 &= 1,0 \rightarrow \text{bit } 1, \text{ sobra } 0 \end{aligned}$$

Então:

$$0,25_{10} = 0,01_2$$

Resultado:

$$6,25_{10} = 110,01_2$$

4.3.5 Nem toda fração decimal termina em binário

Algumas frações decimais têm representação binária infinita.

Exemplo clássico:

$$0,1_{10}$$

Em binário, ele não termina de forma exata com poucos bits:

$$0,1_{10} \approx 0,0001100110011\dots_2$$

Esse ponto é importante em computação porque explica por que alguns números decimais com vírgula podem ter pequenas aproximações quando armazenados em binário.

Para esta parte da prova, o mais importante é dominar:

binário com vírgula $\rightarrow$ decimal
decimal com fração simples $\rightarrow$ binário

5. Binário e hexadecimal

O hexadecimal usa 16 símbolos:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Tabela:

Decimal	Hex	Binário
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

A regra é:

1 dígito hexadecimal = 4 bits

Exemplo:

2F₁₆

Converta cada dígito:

2 = 0010
F = 1111

Logo:

2F₁₆ = 0010 1111₂

Outro exemplo:

$1011\ 0110_2$

Separe em grupos de 4:

$1011 = B$

$0110 = 6$

Logo:

$1011\ 0110_2 = B6_{16}$

6. Soma binária

A soma binária funciona como a soma decimal, mas usando base 2.

6.1 Regras básicas

Operação	Resultado
$0 + 0$	0
$0 + 1$	1
$1 + 0$	1
$1 + 1$	10

O caso mais importante é:

$1 + 1 = 10_2$

Isso significa:

escreve 0 e vai 1

6.2 Exemplo de soma

Somar:

$1011 + 0110$

Montando:

```
 1011
+ 0110
-----
```

Da direita para a esquerda:

```
1 + 0 = 1
1 + 1 = 10 → escreve 0 e vai 1
0 + 1 + 1 = 10 → escreve 0 e vai 1
1 + 0 + 1 = 10 → escreve 0 e vai 1
```

Resultado:

```
 1011
+ 0110
-----
10001
```

Conferindo em decimal:

```
10112 = 11
01102 = 6
100012 = 17
```

Logo:

```
11 + 6 = 17
```

6.3 Soma com quantidade fixa de bits

Em computadores, muitas vezes a quantidade de bits é fixa.

Exemplo usando **4 bits**:

```
1111 + 0001
```

Soma:

```
 1111
+ 0001
-----
1 0000
```

Mas se o sistema usa apenas 4 bits, o resultado armazenado é:

```
0000
```

O 1 extra da esquerda foi perdido.

Isso é importante para entender **overflow**.

7. Subtração binária

A subtração binária também pode ser feita diretamente.

7.1 Regras básicas

Operação	Resultado
$0 - 0$	0
$1 - 0$	1
$1 - 1$	0
$0 - 1$	precisa pegar emprestado

Exemplo:

```
  1100
- 0101
-----
  0111
```

Conferindo:

```
11002 = 12
01012 = 5
01112 = 7
```

Logo:

```
12 - 5 = 7
```

7.2 Subtração usando soma

Na prática, em sistemas digitais, é muito comum transformar a subtração em soma:

$$A - B = A + (-B)$$

Para fazer isso, precisamos representar $-B$.

É aí que entra o **complemento de 2**.

8. Representação com sinal e sem sinal

A mesma sequência de bits pode significar coisas diferentes.

Exemplo:

1011

Se for um número **sem sinal**, então:

$$1011_2 = 11_{10}$$

Mas se for um número **com sinal em complemento de 2 com 4 bits**, então:

$$1011 = -5$$

Então a questão precisa deixar claro se está usando:

sem sinal

ou

com sinal / complemento de 2

8.1 Números sem sinal

Com n bits sem sinal, o intervalo é:

$$0 \text{ até } 2^n - 1$$

Exemplo com 4 bits:

$$\begin{aligned} &0 \text{ até } 2^4 - 1 \\ &0 \text{ até } 16 - 1 \\ &0 \text{ até } 15 \end{aligned}$$

Tabela:

Binário	Valor sem sinal
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

8.2 Números com sinal em complemento de 2

Com n bits em complemento de 2, o intervalo é:

-2^{n-1} até $2^{n-1} - 1$

Exemplo com 4 bits:

-2^3 até $2^3 - 1$
 -8 até $+7$

Tabela:

Binário	Valor em complemento de 2
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	-8
1001	-7
1010	-6
1011	-5
1100	-4
1101	-3
1110	-2
1111	-1

8.3 O bit mais à esquerda

Em complemento de 2:

bit mais à esquerda = 0 → positivo ou zero
bit mais à esquerda = 1 → negativo

Exemplos em 4 bits:

0101 = +5
1011 = -5
1111 = -1
1000 = -8

Mas cuidado:

O primeiro bit não é apenas um “sinal separado”. Ele faz parte do valor.

Em 4 bits com complemento de 2, os pesos são:

-8 4 2 1

Exemplo:

1011

Cálculo:

$$\begin{aligned} &1 \times (-8) + 0 \times 4 + 1 \times 2 + 1 \times 1 \\ &= -8 + 2 + 1 \\ &= -5 \end{aligned}$$

9. Complemento de 1

O **complemento de 1** é obtido invertendo todos os bits:

0 vira 1
1 vira 0

Exemplo:

0101

Complemento de 1:

1010

Outro exemplo:

00000101

Complemento de 1:

11111010

O complemento de 1 aparece porque ele é um passo intermediário para o complemento de 2.

10. Complemento de 2

O **complemento de 2** é:

complemento de 1 + 1

Ou seja:

1. Inverte todos os bits.
2. Soma 1.

10.1 Exemplo: representar -5 em 4 bits

Comece com **+5**:

0101

Inverta todos os bits:

1010

Some 1:

```
1010
+001
----
1011
```

Logo:

-5 = 1011

10.2 Exemplo: representar -5 em 8 bits

Comece com **+5** em 8 bits:

0000101

Inverta:

1111010

Some 1:

```
11111010
+      1
-----
11111011
```

Logo:

-5 = 11111011

10.3 Como descobrir o valor de um número negativo

Exemplo:

1101

Como começa com 1, em complemento de 2 com 4 bits ele é negativo.

Para descobrir o módulo:

1. Inverta todos os bits.
2. Some 1.
3. Coloque o sinal de menos.

Cálculo:

```
1101
↓ inverte
0010
+ 1
----
0011
```

0011 = 3

Logo:

1101 = -3

11. Por que inverter e somar 1 faz sentido?

Essa é a parte mais importante para entender a lógica.

Com 4 bits, existem apenas 16 combinações:

0000 até 1111

Isso significa que as contas acontecem como se fossem em um ciclo de tamanho 16.

Em 4 bits:

$$2^4 = 16$$

Quando uma soma passa de 15, ela volta para 0 se só guardarmos 4 bits.

Exemplo:

$$1111 + 0001 = 1\ 0000$$

Com 4 bits, guardamos apenas:

$$0000$$

Ou seja:

$$15 + 1 = 0$$

dentro desse ciclo de 4 bits.

11.1 O negativo é o valor que soma com o positivo e dá zero

O negativo de 5 precisa satisfazer:

$$5 + (-5) = 0$$

Em 4 bits, como o ciclo tem tamanho 16, queremos um número que, somado com 5, dê 16:

$$5 + ? = 16$$

Então:

$$? = 11$$

11 em binário de 4 bits:

$$1011$$

Logo:

$$1011 \text{ representa } -5$$

Conferindo:

```
  0101   = +5
+ 1011   = -5
-----
1 0000
```

Guardando só 4 bits:

0000

Resultado:

$5 + (-5) = 0$

11.2 Por que o complemento de 1 precisa do +1?

Com 4 bits, o maior valor é:

$1111 = 15$

Inverter os bits de um número é o mesmo que fazer:

$15 - \text{número}$

Exemplo com 5:

$5 = 0101$

Invertendo:

1010

1010 vale 10.

E:

$15 - 5 = 10$

Mas o negativo correto precisa ser:

$16 - 5 = 11$

Por isso somamos 1:

$10 + 1 = 11$

Então:

```
complemento de 1 =  $2^n - 1 - N$   
complemento de 2 =  $2^n - N$ 
```

No caso de 4 bits:

```
complemento de 1 =  $15 - N$   
complemento de 2 =  $16 - N$ 
```

12. Subtração usando complemento de 2

A regra é:

```
 $A - B = A + \text{complemento de 2 de } B$ 
```

12.1 Exemplo: 4 - 2 com 4 bits

Escreva os números:

```
4 = 0100  
2 = 0010
```

Agora faça o complemento de 2 de 2:

```
0010  
↓ inverte  
1101  
+ 1  
----  
1110
```

Então:

```
-2 = 1110
```

Agora some:

```
  0100  
+ 1110  
-----  
1 0010
```

Como usamos 4 bits, descarte o carry extra:

0010

Resultado:

0010 = 2

Logo:

4 - 2 = 2

12.2 Exemplo: 3 - 5 com 4 bits

3 = 0011

5 = 0101

Complemento de 2 de 5:

```
0101
↓ inverte
1010
+ 1
----
1011
```

Então:

-5 = 1011

Agora:

```
0011
+ 1011
-----
1110
```

1110 começa com 1, então é negativo.

Descubra o módulo:

```
1110
↓ inverte
0001
+ 1
----
0010
```

Módulo 2.

Logo:

$1110 = -2$

Resultado:

$3 - 5 = -2$

13. Overflow

13.1 O que é overflow?

Overflow é quando o resultado real de uma operação não cabe na quantidade de bits disponível.

13.2 Overflow sem sinal

Com 4 bits sem sinal:

intervalo = 0 até 15

Exemplo:

$15 + 1 = 16$

Mas 16 não cabe em 4 bits.

Em binário:

```
  1111
+ 0001
-----
1 0000
```

Com 4 bits, sobra:

0000

O resultado armazenado ficou errado.

Logo:

houve overflow

Para números sem sinal, geralmente há overflow quando aparece carry para fora do bit mais significativo.

13.3 Overflow com sinal em complemento de 2

Com 4 bits em complemento de 2:

intervalo = -8 até +7

Exemplo:

$7 + 3 = 10$

Mas 10 não cabe em 4 bits com sinal.

Em binário:

$7 = 0111$

$3 = 0011$

Soma:

```
  0111
+ 0011
-----
 1010
```

1010, em complemento de 2 com 4 bits, vale -6.

Resultado real esperado:

10

Resultado armazenado:

-6

Logo, houve overflow.

13.4 Regra rápida de overflow com sinal

Em complemento de 2, ocorre overflow quando:

positivo + positivo = negativo

ou

negativo + negativo = positivo

Exemplo 1:

$0111 + 0011 = 1010$

Positivo + positivo deu negativo.

Logo:

overflow

Exemplo 2:

$1000 + 1111 = 0111$

Negativo + negativo deu positivo.

Logo:

overflow

Se os sinais são diferentes:

positivo + negativo

então não há overflow aritmético em complemento de 2.

14. Casos especiais

14.1 O zero

Em complemento de 2, só existe um zero:

$0000 = 0$

Se tentar inverter o sinal do zero:

```
0000
↓ inverte
1111
+ 1
----
1 0000
```

Guardando só 4 bits:

0000

Então:

$-0 = 0$

14.2 O menor número negativo

Com 4 bits, o menor valor é:

$1000 = -8$

Se tentar inverter o sinal:

```
1000
↓ inverte
0111
+ 1
----
1000
```

Voltou para 1000.

Isso acontece porque em 4 bits o intervalo é:

-8 até +7

O valor +8 não existe com 4 bits em complemento de 2.

15. Resumo essencial para prova

Base 2

Cada posição vale uma potência de 2.

Binário para decimal

Some os pesos onde o bit é 1.

Decimal para binário

Divida por 2 e leia os restos de baixo para cima.

Parte fracionária

Depois da vírgula, os pesos são $1/2$, $1/4$, $1/8$, $1/16$...

Binário fracionário para decimal:
some os pesos onde o bit é 1.

Decimal fracionário para binário:
multiplique a parte fracionária por 2 e anote as partes inteiras.

Hexadecimal

1 dígito hexadecimal = 4 bits.

Soma binária

$1 + 1 = 10$

Subtração

$A - B = A + (-B)$

Complemento de 1

Inverte todos os bits.

Complemento de 2

Inverte todos os bits e soma 1.

Com sinal em complemento de 2

bit mais à esquerda = 0 → positivo ou zero
bit mais à esquerda = 1 → negativo

Intervalos

n bits sem sinal: 0 até $2^n - 1$

n bits com sinal: -2^{n-1} até $2^{n-1} - 1$

Overflow com sinal

positivo + positivo = negativo → overflow

negativo + negativo = positivo → overflow

16. Exercícios rápidos

Conversão

1. Converta 1011_2 para decimal.
2. Converta 110101_2 para decimal.
3. Converta 13_{10} para binário.
4. Converta 45_{10} para binário.
5. Converta $2F_{16}$ para binário.
6. Converta $1011\ 0110_2$ para hexadecimal.

Soma e subtração

7. Some $1010 + 0011$.
8. Some $0111 + 0011$ em 4 bits.
9. Faça $4 - 2$ usando complemento de 2 com 4 bits.
10. Faça $3 - 5$ usando complemento de 2 com 4 bits.

Sinal

11. Interprete 1011 como número sem sinal.
12. Interprete 1011 como complemento de 2 com 4 bits.
13. Qual o intervalo de 8 bits sem sinal?
14. Qual o intervalo de 8 bits com sinal em complemento de 2?

Complemento de 2

15. Faça -5 em 4 bits.
16. Faça -5 em 8 bits.
17. Descubra o valor de 1110 em complemento de 2 com 4 bits.

18. Descubra o valor de 11111011 em complemento de 2 com 8 bits.

Overflow

19. $0111 + 0011$ em 4 bits com sinal tem overflow?

20. $0101 + 1110$ em 4 bits com sinal tem overflow?

Parte fracionária

21. Converta $101,101_2$ para decimal.

22. Converta $11,01_2$ para decimal.

23. Converta $0,625_{10}$ para binário.

24. Converta $6,25_{10}$ para binário.

25. $0,1_{10}$ tem representação binária finita exata?

26. Converta $101,011_2$ para decimal.

17. Gabarito

- 11
- 53
- 1101
- 101101
- 0010 1111
- B6
- 1101
- 1010, com overflow se for complemento de 2 em 4 bits
- 0010
- 1110, que representa -2
- 11
- 5
- 0 até 255
- 128 até +127
- 1011
- 11111011
- 2
- 5
- Sim
- Não

21. $5,625$

22. $3,25$

23. $0,101_2$

24. $110,01_2$

25. Não. Em binário, $0,1_{10}$ vira uma dízima periódica aproximada: $0,000110011\dots_2$

26. $5,375$